```
:concurrency: <%= ENV.fetch("SIDEKIQ_CONCURRENCY") { 5 } %>
:timeout: <%= ENV.fetch("SIDEKIQ_TIMEOUT_SECONDS") { 25 } %>
» :max_retries: 1
:queues:
  - default
```

Restart your server, then repeat everything. This time, you should see a notification in the error catcher UI with the message you put in the death handler. If you go to the Sidekiq Web UI, click on “Dead,” and you should see the job. This demonstrates that a transient failure will eventually notify you if the job never succeeds.

Don’t forget to remove `max_retries` from your `config/sidekiq.yml`.

The strategy you just implemented—notifying yourself about permanent errors, avoiding notification for transient ones, and then ensuring dead jobs notify you no matter what—forms a solid foundation for managing Sidekiq jobs. This strategy ensures that all jobs eventually succeed without overloading you with non-actionable notifications. Over time, you’ll tweak this setup as you learn which sorts of failures are transient and which are not.

It’s worth pointing out that you really can’t avoid having to handle failures in some way. No paid service can alleviate you of this need, since addressing failure requires understanding the specifics of your app and your jobs. Anyone who tells you different is selling something.